Laboratory work No. 3

«Curve Fitting 20.27»

Submitted by

Farrakhetdinova Azaliya

26/12/2025

Problem statement

The vertical stress σ_z under the corner of a rectangular area subjected to a uniform load of intensity q is given by the solution of Boussinesq's equation:

$$\sigma = \frac{q}{4\pi}\left[\frac{2mn\sqrt{m^2+n^2+1}}{m^2+n^2+1+m^2n^2}\frac{m^2+n^2+2}{m^2+n^2+1} + \sin^{-1}\frac{2mn\sqrt{m^2+n^2+1}}{m^2+n^2+1+m^2n^2}\right]$$

Because this equation is inconvenient to solve manually, it has been reformulated as

$$\sigma_z = q f_z(m,n)$$

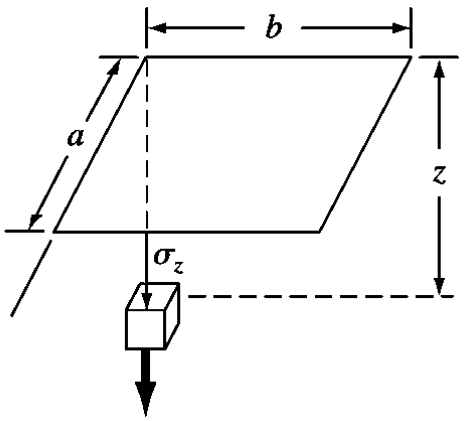

Fig. 1 – Rectangular area

where $f_z(m,n)$ is called the influence value and m and n are dimensionless ratios, with $m = a/z$ and $n = b/z$ and a and b as defined in Fig. 1. The influence value is then tabulated, a portion of which is given in Table 1. If $a = 4.6$ and $b = 14$, use a third-order interpolating polynomial to compute σ_z at a depth $10\ m$ below the corner of a rectangular footing that is subject to a total load of $100\ t$ (metric tons). Express your answer in tonnes per square meter. Note that q is equal to the load per area.

$\boldsymbol{m}$	$\boldsymbol{n = 1.2}$	$\boldsymbol{n = 1.4}$	$\boldsymbol{n = 1.6}$
0.1	0.02926	0.03007	0.03058
0.2	0.05733	0.05894	0.05994
0.3	0.08323	0.08561	0.08709
0.4	0.10631	0.10941	0.11135
0.5	0.12626	0.13003	0.13241
0.6	0.14309	0.14749	0.15027
0.7	0.15703	0.16199	0.16515
0.8	0.16843	0.17389	0.17739

Table 1 – Influence value $f_z(m, n)$

Mathematical model

Mathematical model of the problem consists of the Boussinesq's equation:

$$\sigma = \frac{q}{4\pi}\left[\frac{2mn\sqrt{m^2+n^2+1}}{m^2+n^2+1+m^2n^2}\frac{m^2+n^2+2}{m^2+n^2+1} + \sin^{-1}\frac{2mn\sqrt{m^2+n^2+1}}{m^2+n^2+1+m^2n^2}\right]$$

The simple form of this equation:

$$\sigma_z = qf_z(m,n)$$

where $m = a/z$, $n = b/z$, a, b – sizes of area, z – depth below the corner, q – load per area.

Numerical method

Interpolating function of two variables can be obtain from the representation below:

$$P_{m+n}(x,y) = \sum_{i}^{n}\sum_{j}^{m} a_{ij}x_i y_j$$

Then the polynomial is linear equation with ten coefficients:

$$P_3(x,y) = a_{00} + a_{10}x + a_{01}y + a_{20}x^2 + a_{11}xy + a_{02}y^2 + a_{30}x^3 + a_{21}x^2y + a_{12}xy^2 + a_{03}y^3$$

Now we need obtain polynomial coefficients from system of ten linear equation. For this we use Gauss-elimination method.

This method consists of:

1. Form the Augmented Matrix: Represent the system as an augmented matrix combining coefficients and constants.
2. Forward Elimination: Convert the matrix into row echelon (upper triangular) form.
3. Back-Substitution: Solve the resulting triangular system from the bottom up to find the values of the variables.

Result of numerical solution

The main window of interface is shown on Fig.1. There are the input fields, the solution, and the graph output for visual representation of function $f(m,n)$ (see the axes labels).

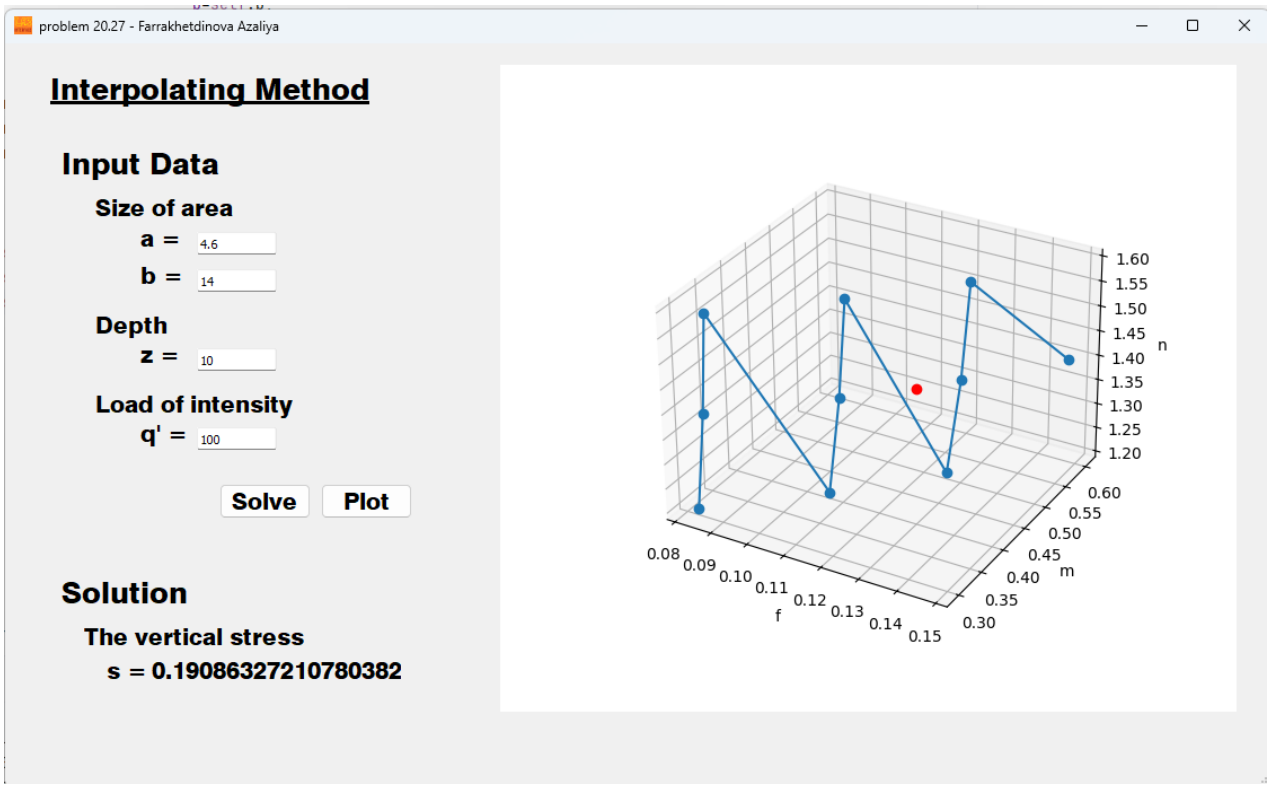

Fig.1 – Main window

Conclusion

For given parameters, the program outputs values of the function $f(m,n)$. We can see at the graph that the interpolation is correct, since value of the $f(m,n)$ for given m and n lies between two neighbor tabulated values.

Appendix.

A. Main Window Code

```
import numpy as np
from PyQt5 import QtWidgets, uic

from solution import Solution

class MainWindow(QtWidgets.QMainWindow):

    def __init__(self):

        super(MainWindow, self).__init__()

        uic.loadUi('interface.ui', self)

        self.solve_button.clicked.connect(self.show_solution)
        self.plot_button.clicked.connect(self.plot_data)

    def plot_data(self):

        self.a = float(self.a_edit.text())
        self.b = float(self.b_edit.text())
        self.z = float(self.z_edit.text())
        self.q = float(self.q_edit.text())

        self.solution = Solution(a=self.a,
                                 b=self.b,
                                 z=self.z,
                                 q=self.q )

        X = np.copy(self.solution.f_arr)
        Y = np.copy(self.solution.m_arr)
        Z = np.copy(self.solution.n_arr)

        self.solution.solve()

        x = self.solution.f_var
        y = self.solution.m_var
        z = self.solution.n_var

        self.widget_graph.canvas.axes.clear()
        self.widget_graph.canvas.axes.plot(X, Y, Z, "-o")
        self.widget_graph.canvas.axes.plot(x, y, z, "ro")
        self.widget_graph.canvas.axes.set_xlabel('f')
        self.widget_graph.canvas.axes.set_ylabel('m')
        self.widget_graph.canvas.axes.set_zlabel('n')
        self.widget_graph.canvas.axes.grid(True)

        self.widget_graph.canvas.draw()

    def show_solution(self):

        self.a = float(self.a_edit.text())
        self.b = float(self.b_edit.text())
        self.z = float(self.z_edit.text())
        self.q = float(self.q_edit.text())

        self.solution = Solution(a=self.a,
                                 b=self.b,
                                 z=self.z,
```

```
                                    q=self.q )


        self.solution.solve()
        sigma_solved = self.solution.sigma
        print('solved')

        self.solution_label.setText(str(sigma_solved))



if __name__ == '__main__':
    app = QtWidgets.QApplication([])
    main = MainWindow()
    main.show()
    app.exec_()
```

B. Gauss-Elimination Method Solver Function

```
import numpy as np

def gauss_elimination(A: np.ndarray,
                      b: np.ndarray):

    n: int = len(b)

    for i in range(n):
        for k in range(i + 1, n):
            m = A[k][i] / A[i][i]
            for j in range(i, n):
                A[k][j] -= m * A[i][j]
            b[k] -= m * b[i]

    x: np.ndarray = np.zeros(shape=n)
    for i in range(n - 1, -1, -1):
        x[i] = b[i]
        for j in range(i + 1, n):
            x[i] -= A[i][j] * x[j]
        x[i] /= A[i][i]

    return x
```

C. Interpolation Functions

```
import numpy as np

def poly_coef(
        m: np.ndarray,
        n: np.ndarray ):

    A: np.ndarray = np.zeros((10, 10))  # по задаче это переменные m и n, по
ним находятся интерполяционные коэффициенты потом
    for k in range(0, 10):
        i = 0
        j = 0
        for l in range(9, -1, -1):
            A[k][l] = pow(m[k], i) * pow(n[k], j)
            if (i + j) < 3:
                i += 1
            elif (j < 3):
                j += 1
                i = 0

    return A

def polynom(
        a: np.ndarray, # по задаче это интерполяционные коэффициенты
        m: float, n: float):

   X: np.ndarray = np.zeros(shape=10, dtype=float) # по задаче это переменные
m и n
   b: float = 0

   i = 0
   j = 0
   for l in range(9, -1, -1):
        X[l] = pow(m, i) * pow(n, j)
        if (i + j) < 3:
            i += 1
        elif (j < 3):
            j += 1
            i = 0

   for k in range(0, 10):
       b += a[k] * X[k]

   return b
```

D. Solution Class

```
import numpy as np

from gauss_method import gauss_elimination
from interpolate_func import polynom, poly_coef

class Solution():

    """ Class for solution problem 20-27 with Interpolation and Gauss-Elimination Method """

    def __init__( self,
                  a: float,
                  b: float,
                  z: float,
                  q: float ):

        self.a: float = a
        self.b: float = b
        self.z: float = z
        self.q: float = q / (self.a * self.b)

        x: np.ndarray = np.zeros(shape=10, dtype=float)

        self.m_var: float
        self.n_var: float
        self.f_var: float
        self.sigma: float

        self.f_arr: np.ndarray = np.array([
            0.08323, # m=0.3 n=1.2
            0.08561, # m=0.3 n=1.4
            0.08709, # m=0.3 n=1.6
            0.10631, # m=0.4 n=1.2
            0.10941, # m=0.4 n=1.4
            0.11135, # m=0.4 n=1.6
            0.12626, # m=0.5 n=1.2
            0.13003, # m=0.5 n=1.4
            0.13241, # m=0.5 n=1.6
            0.14749 # m=0.6 n=1.4
        ])

        self.m_arr: np.ndarray = np.array([
            0.3,
            0.3,
            0.3,
            0.4,
            0.4,
            0.4,
            0.5,
            0.5,
            0.5,
            0.6
        ])

        self.n_arr: np.ndarray = np.array([
            1.2,
            1.4,
            1.6,
            1.2,
            1.4,
```

```
            1.6,
            1.2,
            1.4,
            1.6,
            1.4
        ])

        print("start poly_coef solving")
        self.A: np.ndarray = poly_coef(self.m_arr, self.n_arr)
        print("poly_coef solved")
        print(self.A, "\n\n")

    def solve(self):

        self.x = gauss_elimination(self.A, self.f_arr)

        self.m_var: float = self.a / self.z
        self.n_var: float = self.b / self.z

        self.f_var = polynom(self.x, self.m_var, self.n_var)
        print(self.f_var)

        self.sigma = self.q * self.f_var
```

D. Plotting Class (using Matplotlib)

```
import matplotlib
matplotlib.use('Qt5Agg')

from PyQt5 import QtWidgets

from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure


class MplCanvas(FigureCanvasQTAgg):

    def __init__(self):
        fig = Figure()
        self.axes = fig.add_subplot(111, projection='3d')
        super().__init__(fig)

class MplWidget(QtWidgets.QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.canvas = MplCanvas()
        self.layout = QtWidgets.QVBoxLayout()
        self.layout.addWidget(self.canvas)
        self.setLayout(self.layout)
```